01 / SPRINT 1

Guía de estudio y defensa

App de reconocimiento facial

Entender el proyecto, recorrer el código y explicar el Sprint 1 con tus propias palabras.

EL PROYECTO EN UNA FRASE

El sistema recibe una imagen, obtiene un vector facial, lo compara con los perfiles del personal activo y registra la decisión. El panel administra los datos que hacen posible ese recorrido.

Cómo usar esta guía

- **Primera lectura:** seguí el recorrido completo de las páginas 2 a 7.
- **Segunda lectura:** abrí el código y ubicá las funciones mencionadas.
- **Antes de exponer:** repasá el estado real del sprint, el guion y las preguntas.
- **Para comprobar que entendiste:** resolvé los ejercicios sin mirar las respuestas.

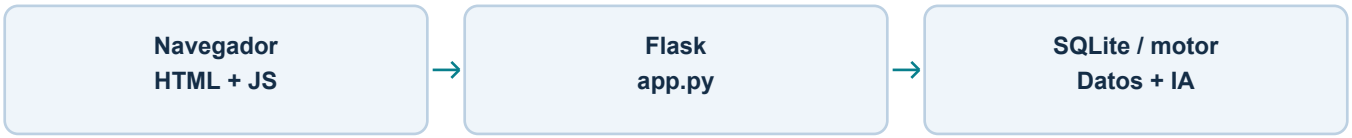
Contenido	Página
Arquitectura, archivos y rutas	2
Administración y estados del personal	3
Enrolamiento y embeddings	4
SQLite y relaciones entre tablas	5
Comparación y decisión de acceso	6
Cámara, formularios y errores	7
Qué se probó y qué queda pendiente	8
Guion para cinco integrantes	9
Preguntas frecuentes de la defensa	10
Ejercicios con respuestas	11
Repaso rápido y mapa de código	12

Versión: 5 de octubre de 2026. Basada en el código local y en la verificación del 2 de octubre. No se ejecutaron nuevas pruebas funcionales al crear esta guía.

02 / SPRINT 1

Arquitectura: quién hace qué

La aplicación separa la presentación, la coordinación del flujo, la persistencia y el procesamiento facial. Una acción en el navegador llega a una ruta de Flask. Esa ruta llama a los módulos necesarios y devuelve una página.



Archivo o carpeta	Responsabilidad
app.py	Rutas, validaciones, carga de archivos, cambios de estado y registros de acceso.
database.py	Conectar con SQLite, crear tablas y ejecutar consultas.
recognition_engine.py	Cargar el modelo, extraer embeddings y buscar la mejor coincidencia.
templates/	Páginas HTML con Jinja; muestran datos enviados por Flask.
static/style.css	Colores, distribución y adaptación de la interfaz a pantallas pequeñas.
data/ y uploads/	Base local y fotografías guardadas. Se excluyen de Git.

Las rutas principales

Ruta	Método y operación
/	GET: métricas y últimos ocho accesos.
/employees	GET: listado de personal y estado de biometría.
/employees/new	GET: formulario. POST: registrar persona.
/employees/<id>/enroll	GET: formulario. POST: crear o actualizar biometría.
/employees/<id>/toggle-status	POST: alternar activo e inactivo.
/access	GET: mostrar molinete. POST: verificar imagen.

IDEA PARA RECORDAR

GET consulta o muestra información. **POST** envía datos para realizar una operación. Una ruta es la dirección que conecta la solicitud del navegador con una función de Flask.

Para decirlo: *"Flask coordina el flujo; el motor facial calcula la coincidencia; SQLite conserva los datos; los templates muestran el resultado".*

03 / SPRINT 1

Administración y estados del personal

Qué ocurre al guardar una persona

• El formulario envía nombre, correo opcional, estado y una foto opcional.

• **new_employee()** lee los datos; **.strip()** quita espacios exteriores.

• Rechaza el nombre vacío y los estados distintos de **active** e **inactive**.

• Inserta el empleado en SQLite y obtiene su ID con **cursor.lastrowid**.

• Si hay foto, intenta crear la biometría. Si no hay foto, el empleado queda registrado con enrolamiento pendiente.

```
full_name = request.form.get("full_name", "").strip()
email = request.form.get("email", "").strip() or None
photo = request.files.get("photo")
```

request.form lee texto del formulario. **request.files** lee archivos. El correo vacío se guarda como NULL; si se informa, la base exige que sea único.

Tres condiciones que no son equivalentes

Condición	Significado	¿Alcanza para ingresar?
Registrado	Existe el empleado en la base.	No.
Activo	Está habilitado para ser considerado en la búsqueda.	No: necesita biometría.
Enrolado	Tiene un vector facial guardado.	No: necesita estar activo y coincidir.

Activación y desactivación

toggle_status() busca al empleado, alterna su estado y actualiza **deactivated_at**. Al desactivarlo registra una fecha; al reactivarlo la limpia. No elimina al empleado ni su perfil facial.

REGLA DE NEGOCIO

Un empleado inactivo queda excluido de la búsqueda de candidatos autorizados. La desactivación impide usar su propio perfil para obtener autorización; no equivale a borrar sus datos.

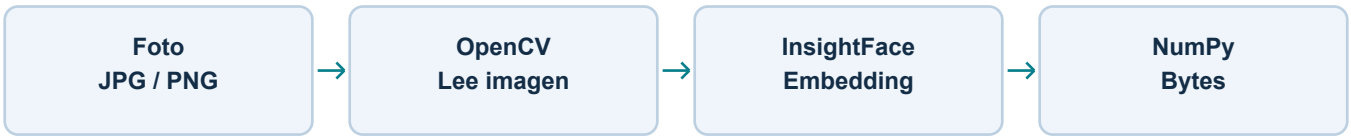
Caso concreto: Ana se registra sin foto. Aparece en Personal, pero todavía no puede ser reconocida. Después carga su biometría. Si la desactivan, deja de ser candidata autorizada.

Matiz: ante un error controlado al cargar la foto, la app conserva el empleado para completar el enrolamiento después. El alta y la biometría no son una única transacción atómica.

04 / SPRINT 1

Enrolamiento: de foto a vector

Enrolar es asociar una representación facial a un empleado. La función **save_biometric_profile()** valida y guarda el archivo, llama al motor y actualiza el perfil en SQLite.



El recorrido del código

• La app comprueba que haya archivo y que su extensión sea JPG, JPEG o PNG.

• Usa el ID, una fecha y **secure_filename()** para construir el nombre dentro de uploads/.

• **cv2.imread()** intenta leer el contenido. Si no puede, se rechaza la imagen.

• **_one_embedding()** exige exactamente un rostro y obtiene **normed_embedding**.

• **np.save(..., allow_pickle=False)** serializa el vector; la app lo guarda como BLOB.

```
faces = _get_model().get(image)
if len(faces) != 1:
    raise ValueError("La imagen debe contener exactamente un rostro.")
return faces[0].normed_embedding.astype(np.float32)
```

Qué es un embedding

Es un vector numérico producido por un modelo entrenado. Sirve para comparar características faciales. Dos fotos de la misma persona pueden generar vectores diferentes, pero suficientemente parecidos.

CUATRO CONCEPTOS

Detección: localizar un rostro en la imagen.
Reconocimiento: compararlo con perfiles conocidos.
Embedding: representación numérica para comparar.
Hash: resultado de otra clase de transformación; no es lo que guarda esta app.

El modelo y los archivos

La configuración usa **FaceAnalysis(name="buffalo_l", providers=["CPUExecutionProvider"])**. ONNX Runtime ejecuta los modelos en CPU. **_model** se carga de forma diferida y se reutiliza por proceso una vez preparado.

La foto queda en uploads/ cuando el enrolamiento funciona. Si falla su guardado o la extracción, se intenta borrar el archivo recién usado. Actualizar el perfil reemplaza el vector, pero actualmente conserva la foto anterior.

Para decirlo: *"Guardamos un embedding facial serializado; no un hash. El modelo ya está entrenado: nuestra app lo utiliza, no lo entrena desde cero".*

05 / SPRINT 1

SQLite: datos y relaciones

SQLite guarda la información en **data/facial_access.db**. Para este prototipo evita configurar un servidor de base de datos. Persistir significa que los datos sobreviven al reinicio de la aplicación.

Tabla	Datos principales	Relación
employees	id, full_name, email, status, created_at, deactivated_at.	Una persona tiene cero o un perfil.
biometric_profiles	employee_id, face_encoding, source_image_path, enrolled_at.	employee_id es único y referencia al empleado.
access_logs	employee_id, device_name, result, detail, created_at.	Una persona puede tener muchos accesos; un intento desconocido puede tener employee_id NULL.

Cómo se conserva la coherencia

• **PRIMARY KEY:** identifica cada registro mediante un ID.

• **UNIQUE:** evita correos repetidos y varios perfiles para un mismo empleado.

• **FOREIGN KEY:** relaciona el perfil y los accesos con un empleado válido.

• **CHECK:** restringe los estados permitidos en la base.

• **PRAGMA foreign_keys = ON:** activa el control de esas relaciones en cada conexión.

Consultas parametrizadas

```
db.execute(
    "UPDATE employees SET status = ? WHERE id = ?",
    (new_status, employee_id),
)
```

Los valores se vinculan como parámetros y no se concatenan dentro del SQL. Esto evita que esos valores se interpreten como instrucciones SQL.

Actualizar sin duplicar el perfil

El guardado utiliza **INSERT ... ON CONFLICT(employee_id) DO UPDATE**. Si el empleado ya tiene perfil, actualiza el vector, la ruta y la fecha; si no, crea el perfil.

CORRECCIÓN IMPORTANTE PARA LA DEFENSA

with connection confirma o revierte la transacción al salir. **No cierra automáticamente la conexión.** En esta versión falta cerrar las conexiones explícitamente.

BLOB es almacenamiento binario. La base contiene los bytes del vector y la ruta de la foto, no el archivo JPG/PNG. **allow_pickle=False** evita cargar objetos Python mediante pickle; no cifra el vector.

06 / SPRINT 1

Cómo se decide el acceso

access() recibe la imagen, selecciona los perfiles activos con biometría y llama a **find_match()**. Es una búsqueda de identificación: compara al candidato contra todos los perfiles seleccionados, no contra un único ID declarado.

```
WHERE employees.status = 'active'
  AND biometric_profiles.face_encoding IS NOT NULL
```

La comparación, paso a paso

• **encoding_from_bytes()** decodifica la imagen y genera el vector candidato.

• **np.load(..., allow_pickle=False)** recupera los vectores registrados.

• El producto **np.asarray(stored) @ candidate** calcula las similitudes.

• **np.argmax()** elige la posición con mayor similitud.

• Si esa similitud es al menos **0,40**, devuelve el ID; si no, devuelve None.

```
similarities = np.asarray(stored) @ candidate
position = int(np.argmax(similarities))
similarity = float(similarities[position])
distance = 1.0 - similarity
```

Los vectores están normalizados: su longitud es uno. En ese caso, su producto punto equivale a la similitud coseno. Mayor similitud implica mayor parecido en el espacio del modelo. La pantalla muestra **distancia = 1 - similitud**: una distancia menor corresponde a mayor similitud.

Ejemplo ilustrativo, no medición del proyecto

Perfil activo	Similitud	Resultado de la búsqueda
Ana	0,78	Mayor valor: supera 0,40; devuelve su ID.
Bruno	0,22	No es el mejor candidato.
Otra imagen: mejor valor	0,31	No supera 0,40; no identifica un autorizado.

IDEA PARA RECORDAR

El umbral 0,40 no significa “40 % de certeza”. Es un corte configurado. Subirlo suele reducir aceptaciones y aumentar rechazos; bajarlo produce el efecto contrario. Su calidad debe medirse con casos reales.

Una coincidencia suficiente genera **allowed**; si no existe, **denied**. El resultado se guarda con **write_access_log()**. Si no hay perfiles, se devuelve (None, None), pero el código procesa primero la imagen: puede fallar antes de llegar a esa devolución.

07 / SPRINT 1

Navegador, cámara y errores

Dos formas de enviar una imagen

Foto subida	Captura en vivo
El input type="file" permite elegir una foto. capture="user" sugiere usar la cámara frontal en dispositivos compatibles.	JavaScript solicita cámara mediante getUserMedia y muestra el video.
multipart/form-data transporta el archivo. Flask lo obtiene desde request.files.	Canvas toma un cuadro del video. toDataURL genera una URL de datos que contiene la imagen en Base64.
El servidor lee los bytes con photo.read().	El servidor decodifica el texto con base64.b64decode().

EVITÁ CONFUNDIR ESTOS PASOS

Base64 representa bytes mediante texto. No cifra la foto, no genera el embedding y no autentica a la persona.

Qué controles existen

• El input **accept** orienta la selección; no garantiza que el archivo tenga contenido válido.

• El servidor revisa la extensión y OpenCV intenta decodificar la imagen.

• **MAX_CONTENT_LENGTH** limita la solicitud completa a 5 MiB, incluido el formulario.

• Se controla la cantidad de rostros y se informa cuando faltan dependencias o no se prepara el modelo.

Qué devuelve Flask

render_template() genera HTML con datos; **redirect()** envía al usuario a otra ruta; **url_for()** construye direcciones; **flash()** prepara mensajes de éxito o error. Jinja muestra esos datos dentro de los templates.

Límites de este flujo

La cámara depende de permisos y del contexto de seguridad del navegador. En un teléfono conectado por HTTP a una dirección de red local puede no estar disponible; la carga de archivo es una alternativa. La cámara real y el clic antes de que el video esté listo siguen sin comprobarse en nuestro entorno.

El código maneja **ValueError** y **RecognitionUnavailable** en enrolamiento y acceso. Otros fallos pueden terminar en HTTP 500. El rechazo por tamaño devuelve HTTP 413; todavía no hay una página propia para explicarlo.

Para decirlo: *“La interfaz ofrece cámara y archivo. La decisión se toma en el servidor, usando la imagen recibida y los perfiles autorizados”.*

08 / SPRINT 1

Estado real del Sprint 1

Diferenciá **lo implementado**, **lo probado** y **lo pendiente**. Tener una función escrita no demuestra que el flujo completo funcione con personas y dispositivos reales.

Resultados de la revisión del 2 de octubre

Nivel de evidencia	Resultado
Pruebas sin motor facial real	Panel, formularios, altas, rechazo de datos inválidos, cambios de estado, registros e integridad SQLite.
Pruebas con partes simuladas	Guardar/actualizar perfil; rechazar cero o dos rostros; permitir vector coincidente y rechazar desconocido o inactivo.
No verificado en ese entorno	Modelo buffalo_l, precisión con fotos reales, cámara, teléfono e interfaz mediante inspección visual.

Se ejecutaron **34 comprobaciones: 28 satisfactorias y 6 fallidas o con controles ausentes**. Se usaron datos ficticios y una base temporal. Esa cifra no representa precisión facial ni porcentaje de cobertura.

Hallazgos pendientes

• Administración sin login y formularios sin protección CSRF.

• En una base nueva, arrancar mediante importación/Flask CLI no crea las tablas. La ejecución documentada con python app.py sí llama a la inicialización.

• Conexiones SQLite sin cierre explícito.

• Un fallo inesperado del detector puede generar HTTP 500.

• Reemplazar la biometría conserva imágenes anteriores.

Fortalezas que sí pueden explicar

Separación de módulos; SQLite persistente para una demo local; consultas parametrizadas; perfil único por empleado; filtro de estado activo; validación de un solo rostro; registro de decisiones normales; alternativa de carga de foto.

FORMA PRECISA DE PRESENTAR EL AVANCE

“Tenemos implementado el flujo básico. Probamos la administración y la lógica de autorización con datos simulados. Falta validar el reconocimiento con personas reales y corregir los hallazgos antes de ampliar el sistema”.

Antes de un uso real: falta prueba de vida, calibración, autenticación y protección de datos biométricos. La ejecución directa usa debug=True y una clave predeterminada si no se configura otra. No hay integración con un molinete físico.

09 / SPRINT 1

Guion de exposición: cinco áreas

Distribución sugerida para una defensa de 6 a 8 minutos. Cada integrante explica un área del trabajo del equipo. Los bloques sirven como apoyo: conviene entenderlos y expresarlos con palabras propias.

1. Objetivo y arquitectura

“Nuestro Sprint 1 construye un prototipo local de control de acceso. Incluye administración de personal, enrolamiento, verificación y registros. Separamos interfaz, servidor, persistencia y motor facial. Flask coordina el flujo; SQLite guarda la información. Por ahora el molinete es un simulador web”.

2. Administración e interfaz

“El panel muestra métricas y últimos accesos. Podemos registrar una persona con nombre, correo opcional y estado, y completar su biometría después. Validamos nombre y estado; el correo informado es único. Desactivar a alguien conserva sus datos, pero excluye su perfil de la búsqueda autorizada”.

3. Base de datos

“Elegimos SQLite porque el prototipo es local. employees guarda al personal; biometric_profiles, su vector y la ruta de la foto; access_logs, los intentos. Cada empleado puede tener un único perfil biométrico. Las consultas usan parámetros y habilitamos claves foráneas para mantener las relaciones”.

4. Enrolamiento y motor

“OpenCV lee la foto e InsightFace obtiene los rostros. Exigimos exactamente uno y guardamos su embedding normalizado como bytes. No guardamos un hash. Utilizamos un modelo ya entrenado, configurado para CPU, y lo reutilizamos después de cargarlo. La validación con personas reales está pendiente en el entorno revisado”.

5. Decisión, pruebas y cierre

“Comparamos la nueva imagen con los perfiles activos. Seleccionamos la mayor similitud y aplicamos el umbral de 0,40. Registramos la decisión. Probamos el panel, la persistencia y la autorización con vectores simulados. El siguiente paso es validar el modelo y la cámara, y corregir errores y controles ausentes”.

Demostración recomendada

Mostrar panel → registrar persona sin foto → consultar listado → mostrar formulario de enrolamiento → desactivar persona → explicar filtro de activos. Solo demostrar reconocimiento real si antes lo probaron en el equipo de la exposición.

TU APOYO SI TE QUEDÁS EN BLANCO

Para explicar cualquier parte, seguí esta secuencia: **qué recibe** → **qué valida** → **qué función llama** → **qué guarda** → **qué devuelve**. Si recordás ese recorrido, podés responder sin memorizar líneas.

10 / SPRINT 1

Preguntas típicas de la defensa

¿Por qué un embedding y no un hash?

Porque necesitamos medir similitud entre fotos distintas. El embedding es la representación que produce el modelo para esa comparación.

¿Ustedes entrenaron la inteligencia artificial?

No. Integramos un modelo preentrenado y programamos su uso dentro de la app.

¿Dónde están las fotos?

En uploads/. SQLite guarda la ruta y el vector como BLOB. La foto no está dentro de la tabla.

¿Qué pasa si hay dos rostros?

El código rechaza la imagen porque exige exactamente uno, tanto al enrolar como al verificar.

¿Puede entrar alguien activo sin foto?

No mediante su propio perfil: necesita biometría registrada y una coincidencia suficiente.

¿Qué significa 0,40?

Es el umbral de similitud. No es una probabilidad. Todavía debe calibrarse con fotos representativas.

¿La distancia y la similitud son lo mismo?

No. La app calcula distancia como 1 menos similitud: menor distancia significa mayor parecido.

¿Por qué SQLite?

Simplifica el prototipo local y conserva los datos sin un servidor independiente. Una migración futura dependería de la concurrencia y del uso esperado.

¿Cada intento queda registrado?

Las decisiones normales y los errores controlados del flujo sí. Algunos fallos inesperados o rechazos previos todavía no generan un registro.

¿El sistema detecta una foto usada para engañarlo?

No implementa prueba de vida. Acepta imágenes; no demuestra presencia física.

¿Qué sucede si no hay perfiles activos?

Se deniega si se procesa la imagen correctamente. El código obtiene primero su embedding, por lo que un error de imagen o modelo puede ocurrir antes.

¿Está listo para producción?

No. Es un prototipo. Faltan validación real, calibración, autenticación, controles de formularios y mejoras en errores y protección de datos.

11 / SPRINT 1

Práctica: comprobá que entendiste

Respondé primero sin mirar. No busques repetir una definición: explicá qué pasa dentro de esta app.

Seis situaciones

1. Ana está activa, pero todavía no tiene perfil biométrico. ¿Su registro alcanza para autorizarla?

2. Bruno tiene perfil, pero está inactivo. ¿El motor lo incluye entre los candidatos?

3. Las similitudes contra dos perfiles activos son 0,35 y 0,62. ¿Cuál elige? ¿Qué distancia devuelve?

4. El mejor valor es 0,39. ¿Se permite el acceso con el umbral actual?

5. Una persona vuelve a cargar su foto. ¿Se crean dos perfiles? ¿Qué pasa hoy con la foto anterior?

6. Se muestra un mensaje de “faltan librerías”. ¿Es equivalente a haber probado el reconocimiento real?

Respuestas para autocorregirte

1. No. No tiene un vector con el que pueda identificarse como perfil autorizado.

2. No. La consulta filtra por status = active antes de comparar los perfiles.

3. El perfil con 0,62. Supera 0,40 y la distancia es 1 - 0,62 = 0,38.

4. No. 0,39 es menor que 0,40; devuelve que no hubo coincidencia autorizada.

5. No crea dos perfiles: actualiza el existente. Actualmente conserva la foto anterior en disco.

6. No. Solo demuestra que se maneja la ausencia de dependencias. Hay que ejecutar el modelo y probar imágenes reales.

Plan breve de estudio: 60 minutos

Tiempo	Actividad
10 min	Leer arquitectura y dibujar el recorrido completo.
15 min	Recorrer alta y enrolamiento en app.py.
15 min	Leer tablas y find_match(); explicar un ejemplo numérico.
10 min	Resolver los seis casos sin mirar respuestas.
10 min	Ensayar en voz alta el guion y responder tres preguntas.

CRITERIO DE ESTUDIO

Podés dar por comprendido un tema cuando lo explicás con un ejemplo, encontrás la función correspondiente y distinguís qué está probado de qué está pendiente.

12 / SPRINT 1

Repaso final y mapa del código

El recorrido que tenés que poder contar

IDEA PARA RECORDAR

Alta: formulario → Flask valida → employees.
Enrolamiento: foto → un rostro → embedding → biometric_profiles.
Acceso: nueva imagen → embedding → perfiles activos → mayor similitud → umbral → access_logs.

Concepto	Recordatorio de una línea
Embedding	Vector para comparar rostros; no es hash ni cifrado.
Normalización	Vector de longitud uno; habilita similitud coseno por producto punto.
BLOB	Datos binarios guardados en SQLite.
Base64	Bytes representados como texto para transportar la captura.
Enrolamiento	Registrar la representación facial de una persona.
Falso positivo / negativo	Autorizar por error / rechazar por error; deben medirse con pruebas reales.
Transacción	Grupo de cambios que se confirma o revierte. No implica cierre de conexión.

Dónde buscar al estudiar

Archivo	Funciones o elementos clave
app.py	dashboard, new_employee, save_biometric_profile, toggle_status, enroll_employee, access, write_access_log.
database.py	get_connection, initialize_database, query_all, query_one; esquema de las tres tablas.
recognition_engine.py	_get_model, _one_embedding, encoding_from_image, encoding_from_bytes, find_match.
templates/access.html	getUserMedia, canvas, snapshot, formulario y carga alternativa.

Frases que conviene evitar

“Hasheamos caras”; “0,40 es 40 % de certeza”; “entrenamos el modelo”; “Base64 cifra la imagen”; “with cierra SQLite”; “ya funciona todo”. Reemplazalas por las explicaciones precisas de esta guía.

Fuentes de esta guía: app.py, database.py, recognition_engine.py, templates/, README.md y VERIFICACION.md del proyecto local. Comportamiento del contexto SQLite: documentación oficial de Python.

Alcance: material de estudio del Sprint 1, preparado el 5 de octubre de 2026. Refleja la implementación leída y la auditoría previa; no certifica precisión biométrica ni aptitud para un control de acceso real.